ΜΑΘΗΜΑ 4 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.2

Σύνολα και λεξικά

Μοναδικά στοιχεία και αναζήτηση με «κλειδί»

Θεωρία σε 12 μικρά βήματα + 17 ασκήσεις στο εργαστήριο Διάρκεια: 4 ώρες (3η–6η) 24 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο · Δευτέρα 26/10/2026

Τι θα ξέρεις στο τέλος

1. Να χρησιμοποιείς **σύνολο** για να κρατάς κάθε τιμή **μία φορά** (χωρίς διπλά).
2. Να βρίσκεις τι είναι **κοινό** σε δύο σύνολα και τι έχει **μόνο** το ένα.
3. Να φτιάχνεις **λεξικό** και να βρίσκεις μια τιμή από το **κλειδί** της (όνομα → τηλέφωνο).
4. Να **προσθέτεις**, να **αλλάζεις** και να **διαγράφεις** στοιχεία ενός λεξικού.
5. Να **μετράς** πόσες φορές εμφανίζεται κάτι, με λεξικό.

Πώς δουλεύουμε

Η θεωρία είναι χωρισμένη σε **12 μικρά βήματα**. Κάθε βήμα έχει πάνω δεξιά την ετικέτα **ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ** με τον αριθμό της άσκησης στο site.
Διαβάζεις τη διαφάνεια → λύνεις την άσκηση → πας στο επόμενο βήμα.

Από το εύκολο στο δύσκολο

Ασκήσεις **1–8**: εύκολες, μία εντολή τη φορά.
Ασκήσεις **9–13**: μέτριες, συνδυάζουν 2–3 βήματα.
Ασκήσεις **14–16**: δύσκολες, όλα μαζί. Η **17** είναι για το σπίτι.

ΣΥΝΔΕΣΗ Οι λίστες βρίσκουν στοιχεία με τη **θέση** τους. Σήμερα: το σύνολο χωρίς θέσεις και το λεξικό με **όνομα** (κλειδί).

Θυμάσαι από την προηγούμενη φορά;

ΕΚΦΩΝΗΣΗ

Πέντε γρήγορες ερωτήσεις, 5 λεπτά. Τι δίνει κάθε κομμάτι; Πρώτα σκέψου, μετά δες τη λύση.

ΛΥΣΗ

ΚΩΔΙΚΑΣ	ΑΠΟΤΕΛΕΣΜΑ	ΓΙΑΤΙ
`nums = [5, 2, 5]; len(nums)`	`3`	μετράει και τα διπλά
`nums[0]`	`5`	η θέση 0 = το πρώτο
`"Άννα" in ["Άννα", "Νίκος"]`	`True`	υπάρχει μέσα
`nums.append(7)`	`[5, 2, 5, 7]`	στο τέλος
`sorted(["Νίκος", "Άννα"])`	`['Άννα', 'Νίκος']`	αλφαβητικά

Γιατί αυτά

Το **len**, το **in**, το **for** και το **sorted** δουλεύουν **ίδια** στα σύνολα και στα λεξικά. Ό,τι ξέρεις, το ξαναχρησιμοποιείς.

ΠΑΡΑΤΗΡΗΣΗ Στη λίστα το 5 μετράει **δύο φορές**. Σε λίγο θα δούμε μια δομή που το κρατά **μία**.

Δύο ερωτήσεις που η λίστα απαντά άβολα

«ΤΟ ΤΗΛΕΦΩΝΟ ΤΟΥ ΝΙΚΟΥ;» · ΜΕ ΛΙΣΤΕΣ

```
names = ["Άννα", "Νίκος"]
phones = ["6971111111", "6972222222"]

i = names.index("Νίκος")
print(phones[i])
```

ΜΕ ΛΕΞΙΚΟ

```
phone = {"Άννα": "6971111111",
         "Νίκος": "6972222222"}

print(phone["Νίκος"])
6972222222
```

«ΠΟΣΟΙ ΔΙΑΦΟΡΕΤΙΚΟΙ;» · ΜΕ ΛΙΣΤΑ

```
visits = ["Άννα", "Νίκος", "Άννα"]
unique = []
for v in visits:
    if v not in unique:
        unique.append(v)
print(len(unique))
```

ΜΕ ΣΥΝΟΛΟ

```
visits = ["Άννα", "Νίκος", "Άννα"]
print(len(set(visits)))
2
```

Η ΙΔΕΑ Κάθε δομή είναι φτιαγμένη για ένα είδος ερώτησης. Με τη σωστή, ο κώδικας γίνεται **πιο μικρός**.

Σύνολο: κάθε τιμή μία φορά

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 1

```
visits = ["Άννα", "Νίκος", "Άννα", "Ελένη"]
people = set(visits)
print(people)
{'Νίκος', 'Άννα', 'Ελένη'}
print(len(people))
3
print(sorted(people))
['Άννα', 'Ελένη', 'Νίκος']

colors = {"κόκκινο", "μπλε"}
```

set(λίστα)

Φτιάχνει σύνολο από μια λίστα. Τα **διπλά φεύγουν μόνα τους**.

Χωρίς σειρά

Το σύνολο **δεν έχει θέσεις**: το people[0] δεν υπάρχει. Όταν τυπώνεται, η σειρά μπορεί να βγει ανακατεμένη. Για σειρά: **sorted()**.

Άγκιστρα { }

Γράφεται και κατευθείαν με **{ }**. Για **άδειο** σύνολο όμως γράφεις **set()**, όχι {}.

ΠΟΥ ΧΡΗΣΙΜΕΥΕΙ Πόσοι διαφορετικοί επισκέπτες; Πόσες διαφορετικές λέξεις; → **len(set(...))**.

Βάζω και ψάχνω σε σύνολο

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 2

```
stickers = {3, 7, 12}

stickers.add(5)       # νέο
stickers.add(7)       # το έχω ήδη: τίποτα
print(sorted(stickers))
[3, 5, 7, 12]

print(12 in stickers)
True
print(9 in stickers)
False

stickers.remove(3)    # το βγάζω
print(len(stickers))
3
```

add(τιμή)

Βάζει την τιμή στο σύνολο. Αν υπάρχει ήδη, **δεν γίνεται τίποτα** — ούτε σφάλμα.

in

Υπάρχει ή όχι; **True** ή **False**, όπως στη λίστα.

remove(τιμή)

Τη βγάζει. Αν δεν υπάρχει → σφάλμα, οπότε έλεγξε πρώτα με **if … in …**.

ΓΙΑΤΙ ADD ΚΙ ΟΧΙ APPEND Το append σημαίνει «στο **τέλος**». Το σύνολο δεν έχει τέλος, ούτε σειρά.

Τι είναι κοινό, τι είναι μόνο στο ένα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 3

python = {"Άννα", "Νίκος", "Ελένη"} java = {"Νίκος", "Μαρία"}

ΓΡΆΦΕΙΣ	ΠΑΊΡΝΕΙΣ	ΜΕ ΛΌΓΙΑ
`python & java`	`{'Νίκος'}`	**και** στα δύο (κοινά)
`python | java`	`{'Άννα', 'Νίκος', 'Ελένη', 'Μαρία'}`	**όλοι** μαζί
`python - java`	`{'Άννα', 'Ελένη'}`	**μόνο** Python
`java - python`	`{'Μαρία'}`	**μόνο** Java

Πώς τα θυμάμαι

& = «και»
| = «ή»
- = «χωρίς»

```
print(sorted(python & java))
['Νίκος']
```

ΟΠΩΣ ΣΤΑ ΜΑΘΗΜΑΤΙΚΑ Τομή, ένωση, διαφορά. Ζωγράφισε δύο κύκλους που μπλέκονται και βάλε κάθε όνομα στη θέση του.

Πράξεις συνόλων

ΕΚΦΩΝΗΣΗ

Με a = {1, 2, 3, 4} και b = {3, 4, 5}, τι δίνει κάθε έκφραση; Ζωγράφισε πρώτα τους δύο κύκλους.

ΛΥΣΗ

ΈΚΦΡΑΣΗ	ΑΠΟΤΈΛΕΣΜΑ	ΜΕ ΛΌΓΙΑ
`a & b`	`{3, 4}`	κοινά
`a | b`	`{1, 2, 3, 4, 5}`	όλα μαζί
`a - b`	`{1, 2}`	μόνο στο a
`b - a`	`{5}`	μόνο στο b
`len(a | b)`	`5`	πόσα όλα μαζί
`6 in a`	`False`	δεν υπάρχει

Έλεγχος

Μόνο a (2) + κοινά (2) + μόνο b (1) = **5** = όσα έχει το **a | b**. ✓

ΤΕΧΝΙΚΗ Με τους κύκλους οι απαντήσεις βγαίνουν με μια ματιά.

Λεξικό: κλειδί → τιμή

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 4

```
prices = {"καφές": 2.5,
          "τσάι": 2.0,
          "χυμός": 3.2}

print(prices["καφές"])
2.5
print(len(prices))
3
```

ΤΟ PRICES ΣΑΝ ΠΙΝΑΚΑΣ

ΚΛΕΙΔΊ	ΤΙΜΉ
"καφές"	2.5
"τσάι"	2.0
"χυμός"	3.2

Σαν πραγματικό λεξικό

Ψάχνεις τη **λέξη** (κλειδί) και βρίσκεις τον **ορισμό** (τιμή). Δεν σε νοιάζει σε ποια θέση είναι.

Πώς γράφεται

{ κλειδί: τιμή, κλειδί: τιμή } — άγκιστρα, και **άνω-κάτω τελεία** ανάμεσα σε κλειδί και τιμή.

prices["καφές"]

Στις αγκύλες βάζεις το **κλειδί**, όχι θέση. Παίρνεις την τιμή του.

ΠΑΡΑΔΕΙΓΜΑΤΑ όνομα → τηλέφωνο · προϊόν → τιμή · μαθητής → βαθμός · λέξη → πόσες φορές.

Όταν το κλειδί δεν υπάρχει

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 5

ΤΟ ΠΡΟΒΛΗΜΑ

```
print(prices["πίτσα"])
KeyError: 'πίτσα'
```

ΛΥΣΗ 1 · ΕΛΕΓΧΟΣ ΜΕ IN

```
item = input("Προϊόν: ")
if item in prices:
    print(prices[item])
else:
    print("Δεν υπάρχει")
```

ΛΥΣΗ 2 · GET

```
print(prices.get("πίτσα", "Δεν υπάρχει"))
Δεν υπάρχει
```

KeyError

Ζήτησες κλειδί που **δεν υπάρχει** στο λεξικό. Είναι το «IndexError» των λεξικών.

in

Το **"πίτσα" in prices** ρωτάει: υπάρχει αυτό το **κλειδί**;

get(κλειδί, αλλιώς)

Δίνει την τιμή αν υπάρχει το κλειδί, **αλλιώς** τη 2η τιμή που του έδωσες. Ποτέ σφάλμα.

ΔΙΑΛΕΞΕ Και οι δύο λύσεις είναι σωστές. Το **if ... in** είναι πιο καθαρό όταν θες να κάνεις και άλλα πράγματα.

Προσθέτω και αλλάζω

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 6

```
stock = {"μήλα": 10, "αχλάδια": 4}

stock["κεράσια"] = 25    # νέο κλειδί
stock["μήλα"] = 8        # αλλαγή τιμής
stock["αχλάδια"] += 6    # 4 + 6 = 10

print(stock)
{'μήλα': 8, 'αχλάδια': 10, 'κεράσια': 25}
```

Ίδια εντολή, δύο δουλειές

stock[κλειδί] = τιμή: αν το κλειδί **δεν υπάρχει**, το φτιάχνει. Αν **υπάρχει**, αλλάζει την τιμή του.

+= 6

Σημαίνει **stock["αχλάδια"] = stock["αχλάδια"] + 6**. Δουλεύει μόνο αν το κλειδί υπάρχει ήδη.

Κάθε κλειδί μία φορά

Δεν γίνεται να έχεις δύο "μήλα". Η νέα τιμή **αντικαθιστά** την παλιά.

ΘΥΜΗΣΟΥ Στη λίστα έκανες append. Στο λεξικό απλώς γράφεις **d[νέο_κλειδί] = τιμή**.

Διαγράφω ένα κλειδί

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 7

```
stock = {"μήλα": 8, "αχλάδια": 10, "κεράσια": 25}

del stock["κεράσια"]
print(stock)
{'μήλα': 8, 'αχλάδια': 10}
print(len(stock))
2

n = stock.pop("μήλα")    # το βγάζω και το κρατάω
print(n, stock)
8 {'αχλάδια': 10}
```

del d[κλειδί]

Σβήνει το κλειδί **και** την τιμή του.

pop(κλειδί)

Το βγάζει και σου δίνει την τιμή του — όπως το pop της λίστας.

ΣΥΧΝΟ ΛΑΘΟΣ **del stock["πίτσα"]** όταν δεν υπάρχει → KeyError. Έλεγξε πρώτα: **if "πίτσα" in stock:**

Τι έχει το λεξικό μετά από κάθε γραμμή;

ΕΚΦΩΝΗΣΗ

Γράψε στο τετράδιο πώς είναι το λεξικό **μετά από κάθε γραμμή**. Μετά τρέξ' το στο «🧪 Ελεύθερο» του εργαστηρίου και έλεγξε.

ΛΥΣΗ

ΓΡΑΜΜΉ	ΤΟ ΛΕΞΙΚΌ ΜΕΤΆ	ΤΙ ΈΓΙΝΕ
`d = {"x": 1}`	`{'x': 1}`	φτιάχτηκε
`d["y"] = 2`	`{'x': 1, 'y': 2}`	νέο κλειδί
`d["x"] = 5`	`{'x': 5, 'y': 2}`	άλλαξε το x
`d["z"] = d["x"] + d["y"]`	`{'x': 5, 'y': 2, 'z': 7}`	5 + 2
`del d["y"]`	`{'x': 5, 'z': 7}`	έφυγε το y
`print(len(d))`	`2`	δύο κλειδιά

Η παγίδα

Το **d["x"] = 5** δεν φτιάχνει δεύτερο x. **Αλλάζει** το υπάρχον.

ΤΕΧΝΙΚΗ Γράφε ολόκληρο το λεξικό σε κάθε γραμμή, όπως με τις λίστες.

Περνάω από όλο το λεξικό

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 8

1 · ΤΑ ΚΛΕΙΔΙΑ

```
prices = {"καφές": 2.5, "τσάι": 2.0}
for item in prices:
    print(item, prices[item])
καφές 2.5
τσάι 2.0
```

for item in prices

Το for σε λεξικό δίνει τα **κλειδιά**. Την τιμή την παίρνεις με **prices[item]**.

2 · ΚΛΕΙΔΙ ΚΑΙ ΤΙΜΗ ΜΑΖΙ

```
for item, price in prices.items():
    print(f"{item}: {price:.2f}€")
καφές: 2.50€
τσάι: 2.00€
```

.items()

Δίνει **ζευγάρια** (κλειδί, τιμή). Στο for τα μοιράζεις σε δύο μεταβλητές, όπως το **x, y = point**.

ΑΛΦΑΒΗΤΙΚΑ **for item in sorted(prices):** περνά τα κλειδιά με αλφαβητική σειρά.

Άθροισμα, μέσος όρος, ο καλύτερος

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 9

grades = {"Άννα": 17, "Νίκος": 9, "Ελένη": 15}

ΓΡΆΦΕΙΣ	ΠΑΊΡΝΕΙΣ	ΤΙ ΕΊΝΑΙ
`len(grades)`	3	πόσα κλειδιά
`grades.values()`	17, 9, 15	μόνο οι τιμές
`sum(grades.values())`	41	άθροισμα
`max(grades.values())`	17	ο μεγαλύτερος βαθμός

Μέσος όρος

sum(grades.values()) / len(grades) → 13.67

Ο καλύτερος

Κρατάς σε μια μεταβλητή **best** το όνομα του καλύτερου ως τώρα και το αλλάζεις όταν βρεις μεγαλύτερο βαθμό.

ΠΟΙΟΣ ΕΧΕΙ ΤΟΝ ΜΕΓΑΛΥΤΕΡΟ;

```
best = ""
for name in grades:
    if best == "" or grades[name] > grades[best]:
        best = name
print("Καλύτερος/η:", best)
Καλύτερος/η: Άννα
```

ΘΥΜΗΣΟΥ Το max(grades.values()) δίνει τον **βαθμό** (17). Για το **όνομα** χρειάζεσαι τον βρόχο.

Μετράω πόσες φορές με λεξικό

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 10 · 11

1 · ΜΕ IF

```
votes = ["Α", "Β", "Α", "Γ", "Α"]
counts = {}
for v in votes:
    if v in counts:
        counts[v] += 1
    else:
        counts[v] = 1
print(counts)
{'Α': 3, 'Β': 1, 'Γ': 1}
```

2 · ΤΟ ΙΔΙΟ ΜΕ GET, ΣΕ ΜΙΑ ΓΡΑΜΜΗ

```
counts = {}
for v in votes:
    counts[v] = counts.get(v, 0) + 1
print(counts)
{'Α': 3, 'Β': 1, 'Γ': 1}
```

Πώς σκέφτεται

Πρώτη φορά που βλέπω το **"Α"**; → **1**. Το έχω ξαναδεί; → **+1**.

counts.get(v, 0) + 1

«Πόσες φορές το έχω δει ως τώρα (**0** αν πρώτη φορά) **συν 1**». Διάλεξε όποια μορφή καταλαβαίνεις.

ΠΑΝΤΟΥ Ψήφοι, γράμματα μιας λέξης, λέξεις ενός κειμένου, προϊόντα που πουλήθηκαν: **ίδιο μοτίβο**.

Γεμίζω το λεξικό από τον χρήστη

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 12 · 17

```
phone = {}
while True:
    name = input("Όνομα: ")
    if name == "":
        break
    phone[name] = input("Τηλέφωνο: ")

print(len(phone), "επαφές")
for name in sorted(phone):
    print(f"{name}: {phone[name]}")
Όνομα: Νίκος
Τηλέφωνο: 6972222222
Όνομα: Άννα
Τηλέφωνο: 6971111111
Όνομα:
2 επαφές
Άννα: 6971111111
Νίκος: 6972222222
```

Άδειο λεξικό πριν

phone = {} μπαίνει **πριν** τον βρόχο, όπως το **[]** στις λίστες.

while True + break

Ο χρήστης πατά σκέτο **Enter** στο όνομα για να σταματήσει (Μάθημα 3, βήμα 10).

Μία γραμμή για αποθήκευση

phone[name] = input(...): το όνομα γίνεται κλειδί, το τηλέφωνο τιμή.

ΠΡΟΣΟΧΗ Αν δώσεις ξανά το ίδιο όνομα, το παλιό τηλέφωνο **αντικαθίσταται** (βήμα 6).

Κάθε κλειδί, μια λίστα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 13

```
grades = {"Άννα": [17, 15, 18],
          "Νίκος": [9, 11, 8]}

print(grades["Νίκος"])
[9, 11, 8]
print(grades["Νίκος"][0])
9

grades["Άννα"].append(20)      # ένας βαθμός ακόμα
grades["Μαρία"] = [14]         # νέος μαθητής

for name, gl in grades.items():
    print(name, round(sum(gl) / len(gl), 2))
Άννα 17.5
Νίκος 9.33
Μαρία 14.0
```

Ποιο είναι ποιο

Κλειδί = όνομα μαθητή. **Τιμή** = η **λίστα** με τους βαθμούς του.

Σε μια λίστα, ό,τι ξέρεις

Το **grades["Άννα"]** είναι λίστα. Κάνεις ό,τι έκανες στο Μάθημα 3: append, sum, len, [0].

Νέος μαθητής

Του δίνεις λίστα με τον πρώτο βαθμό: **grades["Μαρία"] = [14]**.

ΓΙΑ ΩΡΑΙΑ ΕΞΟΔΟ Με f-string: **f"{name}: {avg:.2f}"** → Νίκος: 9.33

Ποια δομή ταιριάζει;

ΔΟΜΉ	ΓΡΆΦΕΤΑΙ	ΤΗ ΘΈΛΩ ΌΤΑΝ…	ΠΑΡΆΔΕΙΓΜΑ
λίστα	`[1, 2, 2]`	μετράει η σειρά και αλλάζει	βαθμοί που δίνει ο χρήστης
πλειάδα	`(3, 4)`	τιμές που πάνε μαζί, δεν αλλάζουν	θέση (x, y), χρώμα RGB
σύνολο	`{1, 2}`	με νοιάζει μόνο «υπάρχει ή όχι;»	διαφορετικοί επισκέπτες
λεξικό	`{"Άννα": 17}`	ψάχνω με **όνομα** (κλειδί)	όνομα → τηλέφωνο

ΤΙ ΘΑ ΔΙΑΛΕΓΕΣ; · ΛΥΣΗ

ΠΕΡΊΠΤΩΣΗ	ΔΙΑΛΈΓΩ	ΓΙΑΤΊ
α) Τα διαφορετικά hashtags σε 1.000 posts	**σύνολο**	τα διπλά φεύγουν μόνα τους
β) Ο βαθμός κάθε μαθητή, από το όνομα	**λεξικό**	όνομα → βαθμός
γ) Η σειρά των πελατών σε μια ουρά	**λίστα**	η σειρά μετράει, μπαίνουν και βγαίνουν
δ) Πόσες φορές λέγεται κάθε λέξη	**λεξικό**	λέξη → πλήθος (μετρητής)

Η ΕΡΩΤΗΣΗ ΠΟΥ ΑΠΟΦΑΣΙΖΕΙ Με **θέση** το ψάχνω; → λίστα. Με **όνομα**; → λεξικό. Μόνο «**υπάρχει;**» → σύνολο.

Οι 17 ασκήσεις, βήμα-βήμα

#	ΆΣΚΗΣΗ	ΕΠΊΠΕΔΟ	ΔΙΑΦΆΝΕΙΑ
1	Μοναδικοί επισκέπτες	Εύκολο	5
2	Συλλογή αυτοκόλλητων	Εύκολο	6
3	Δύο μαθήματα: κοινά	Εύκολο	7
4	Τιμοκατάλογος	Εύκολο	9
5	Υπάρχει στον κατάλογο;	Εύκολο	10
6	Αποθήκη: βάζω, αλλάζω	Εύκολο	11
7	Αποθήκη: διαγράφω	Εύκολο	12
8	Όλος ο τιμοκατάλογος	Εύκολο	14
9	Μέσος όρος και καλύτερος	Μέτριο	15

#	ΆΣΚΗΣΗ	ΕΠΊΠΕΔΟ	ΔΙΑΦΆΝΕΙΑ
10	Ψηφοφορία	Μέτριο	16
11	Ο μετρητής γραμμάτων	Μέτριο	16
12	Γέμισε τον κατάλογο	Μέτριο	17
13	Λεξικό με λίστες βαθμών	Μέτριο	18
14	Αφαίρεση διπλοτύπων	Δύσκολο	6, 21
15	Συχνότητα λέξεων	Δύσκολο	16, 22
16	Τηλεφωνικός κατάλογος	Ο στόχος	23
17	Ονόματα και συχνότητες	Για το σπίτι	16, 17

ΣΤΟ SITE Εργαστήριο → **Μάθημα 4**. Κάθε άσκηση έχει το «📖 Στις διαφάνειες» που ανοίγει κατευθείαν τη σωστή διαφάνεια.

Αφαίρεση διπλοτύπων

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 14

ΕΚΦΩΝΗΣΗ

Η λίστα emails έχει διπλές εγγραφές (και μία με κεφαλαία). Φτιάξε λίστα **χωρίς διπλά** που κρατά τη **σειρά εγγραφής**. Εμφάνισε πόσα ήταν, πόσα έμειναν και τη λίστα.

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```
emails = ["a@x.gr", "b@y.gr", "A@x.gr", "c@z.gr", "b@y.gr"]
seen = set()
ordered = []

for e in emails:
    e = e.lower()            # A@x.gr → a@x.gr
    if e not in seen:
        seen.add(e)
        ordered.append(e)

print(len(emails), "→", len(ordered))
print(ordered)
```

```
5 → 3
['a@x.gr', 'b@y.gr', 'c@z.gr']
```

Γιατί όχι σκέτο set(emails);

Το set **χάνει τη σειρά**. Το seen θυμάται τι είδαμε, η ordered κρατά τη σειρά.

Από ποια βήματα;

2 add, in · Μάθημα 3: **append**, **for**

ΓΙΑΤΙ .lower() Κάνει όλα τα γράμματα πεζά, ώστε το A@x.gr να μετράει ίδιο με το a@x.gr.

Συχνότητα λέξεων

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 15

ΕΚΦΩΝΗΣΗ

Ο χρήστης γράφει μια πρόταση. Εμφάνισε πόσες φορές λέγεται **κάθε λέξη** (αλφαβητικά) και ποια είναι η **πιο συχνή**. Κεφαλαία και πεζά μετράνε ίδια.

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```
text = input("Κείμενο: ").lower()
words = text.split()          # σπάει σε λέξεις

counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1

for w in sorted(counts):
    print(f"{w}: {counts[w]}")

best = ""
for w in counts:
    if best == "" or counts[w] > counts[best]:
        best = w
print("Πιο συχνή:", best, counts[best])
```

```
Κείμενο: Το παιδί και το σκυλί και το γατί
γατί: 1
και: 2
παιδί: 1
σκυλί: 1
το: 3
Πιο συχνή: το 3
```

Από ποια βήματα;

10 μετρητής · **8** for, sorted · **9** ο καλύτερος

ΔΥΟ ΕΡΓΑΛΕΙΑ ΚΕΙΜΕΝΟΥ **.lower()**: όλα πεζά, ώστε «Το» = «το». **.split()**: σπάει το κείμενο σε λίστα λέξεων.

Τηλεφωνικός κατάλογος

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 16

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```
phone = {}
while True:
    print("\n1.Προσθήκη 2.Αναζήτηση 3.Διαγραφή 4.Όλοι 0.Έξοδος")
    choice = input("Επιλογή: ")
    if choice == "1":
        name = input("Όνομα: ")
        phone[name] = input("Τηλέφωνο: ")
    elif choice == "2":
        name = input("Όνομα: ")
        print(phone.get(name, "Δεν βρέθηκε"))
    elif choice == "3":
        name = input("Όνομα: ")
        if name in phone:
            del phone[name]
            print("Διαγράφηκε")
        else:
            print("Δεν βρέθηκε")
    elif choice == "4":
        for name in sorted(phone):
            print(f"{name}: {phone[name]}")
    elif choice == "0":
        break
    else:
        print("Λάθος επιλογή")
```

Ο σκελετός

Το μενού με **while True** του Μαθήματος 2. Το λεξικό **phone** είναι η «αποθήκη».

Από ποια βήματα;

6 προσθήκη · **5** get · **7** del · **8** for, sorted

Δοκίμασε «κακές» εισόδους

Όνομα που δεν υπάρχει, επιλογή 9. Το πρόγραμμα δεν πρέπει να «σκάει».

ΤΟ ΠΡΟΒΛΗΜΑ ΠΟΥ ΜΕΝΕΙ Μόλις κλείσει το πρόγραμμα, οι επαφές **χάνονται**. Την επόμενη φορά: αποθήκευση σε **αρχείο**.

Τι κρατάμε

ΘΈΛΩ ΝΑ…	ΓΡΆΦΩ
βγάλω τα διπλά	`set(λίστα)`
βάλω σε σύνολο / δω αν υπάρχει	`s.add(x)` · `x in s`
κοινά · όλα · μόνο στο a	`a & b` · `a | b` · `a - b`
φτιάξω λεξικό	`d = {"καφές": 2.5}`
πάρω μια τιμή	`d["καφές"]` · `d.get(k, "–")`
προσθέσω ή αλλάξω	`d["τσάι"] = 2.0`
διαγράψω	`del d["τσάι"]`
περάσω από όλα	`for k, v in d.items():`
μετρήσω	`c[x] = c.get(x, 0) + 1`

Οι 4 δομές

λίστα [ ] · **πλειάδα** ( ) · **σύνολο** { } · **λεξικό** {κλειδί: τιμή}. Τώρα τις ξέρεις όλες!

Επόμενη φορά · Δευ 2/11

Αρχεία και εξαιρέσεις: ο κατάλογος θα αποθηκεύεται σε αρχείο, και τα λάθη του χρήστη δεν θα «ρίχνουν» το πρόγραμμα.

ΓΙΑ ΤΟ ΣΠΙΤΙ Άσκηση **17** στο εργαστήριο: ονόματα μέχρι κενή γραμμή και πόσες φορές το καθένα (βήματα 10–11).